计算机操作系统实验报告：实验4.2 进程同步实验

姓名：王长儒 学号：2412410023 日期：2026年11月4日

一、实验目的

理解进程同步与互斥，掌握信号量的初始化及 P、V
操作，利用共享内存和信号量解决生产者—消费者问题。

二、实验内容与环境

建立一个生产者进程和一个消费者进程，共享容量为 5 的循环缓冲区；生产者依次放入 20
个整数，消费者按先进先出顺序取出。目标环境为支持 POSIX 进程间信号量的 Linux 系统，使用 GCC
编译。

三、设计与原理

使用 mmap 创建共享内存，父子进程共享缓冲区、读写下标和信号量。empty 初值为 5，表示空槽数；full
初值为 0，表示已有产品数；mutex 初值为 1，保护缓冲区和下标。sem_init 的 pshared 参数为
1，使信号量可在进程间共享。

生产者依次执行 P(empty)、P(mutex)、放入产品、V(mutex)、V(full)；消费者依次执行 P(full)、P(mut
ex)、取出产品、V(mutex)、V(empty)。先等待资源再进入临界区，避免进程持有互斥锁等待对方释放资源
而死锁。读写下标对 5 取模，实现循环使用。

四、程序与运行方法

将以下代码保存为 producer_consumer.c，在 Linux 终端依次执行：

```
gcc -std=c11 -Wall -Wextra -O2 producer_consumer.c -pthread -o pc
./pc
```

程序输出每次放入、取出的产品及槽位；正常结束时应显示 PASS，并且生产和消费各 20 次，消费顺序为
1 至 20。并发调度可能改变输出交错顺序。

五、同步机制分析

full 在产品放入后增加，消费者先等待 full，因此不会读取尚未生产的产品；empty
在产品取出后增加，生产者先等待 empty，因此不会覆盖尚未消费的产品。mutex
保证缓冲区及下标的修改互斥。

六、实验思考

1. 多产品：采用固定大小的循环缓冲区及 empty、full 两个计数信号量，可连续生产和消费多个产品。

2. 进程与线程：两者均可并发执行，访问共享资源时均需同步；进程拥有相互隔离的地址空间，需借助共
享内存等机制交换数据，线程通常共享所属进程的地址空间，通信方便但更容易发生数据竞争。

3. 读者—写者：使用计数器记录当前读者数，并用互斥信号量保护计数器；首位读者获取资源锁，末位读
者释放资源锁，写者独占资源锁。多个读者可以同时读取。以下伪代码展示读者优先方案，持续有读者到
来时写者可能饥饿。

```
reader: P(count_mutex); count++;
        if count == 1: P(resource); V(count_mutex);
```

```
        read();
        P(count_mutex); count--;
        if count == 0: V(resource); V(count_mutex);
writer: P(resource); write(); V(resource);
```

上述伪代码中 V(count_mutex) 在条件判断结束后执行，不属于 if 的条件体。初始 count=0，count_mutex=1，resource=1。

附录：源程序

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <semaphore.h>
#include <errno.h>
#define N 5
#define TOTAL 20
typedef struct { sem_t empty, full, mutex; int a[N], in, out; } Shared;
void P(sem_t *s) { while (sem_wait(s) == -1) { if (errno != EINTR) { perror("sem_wait"); _exit(1); } } }
void V(sem_t *s) { if (sem_post(s) == -1) { perror("sem_post"); _exit(1); } }
int main(void) {
    Shared *s = mmap(NULL,sizeof *s,PROT_READ|PROT_WRITE,MAP_SHARED|MAP_ANONYMOUS,-1,0);
    if (s == MAP_FAILED) { perror("mmap"); return 1; }
    s->in=s->out=0;
    if (sem_init(&s->empty,1,N)||sem_init(&s->full,1,0)||sem_init(&s->mutex,1,1)) { perror("sem_init"); return 1; }
    setbuf(stdout,NULL);
    pid_t p=fork();
    if (p<0) { perror("fork"); return 1; }
    if (p==0) {
        for(int i=1;i<=TOTAL;i++) {
            P(&s->empty); P(&s->mutex);
            s->a[s->in]=i; printf("produce %d at %d\n",i,s->in); s->in=(s->in+1)%N;
            V(&s->mutex); V(&s->full);
        }
        _exit(0);
    }
    for(int i=1;i<=TOTAL;i++) {
        P(&s->full); P(&s->mutex);
        int x=s->a[s->out]; printf("consume %d at %d\n",x,s->out); s->out=(s->out+1)%N;
        V(&s->mutex); V(&s->empty);
        if(x!=i) { fprintf(stderr,"order error\n"); return 1; }
    }
    int status; waitpid(p,&status,0);
    sem_destroy(&s->empty); sem_destroy(&s->full); sem_destroy(&s->mutex); munmap(s,sizeof *s);
    if (!WIFEXITED(status)||WEXITSTATUS(status)) return 1;
    puts("PASS: 20 items consumed in FIFO order"); return 0;
}
```